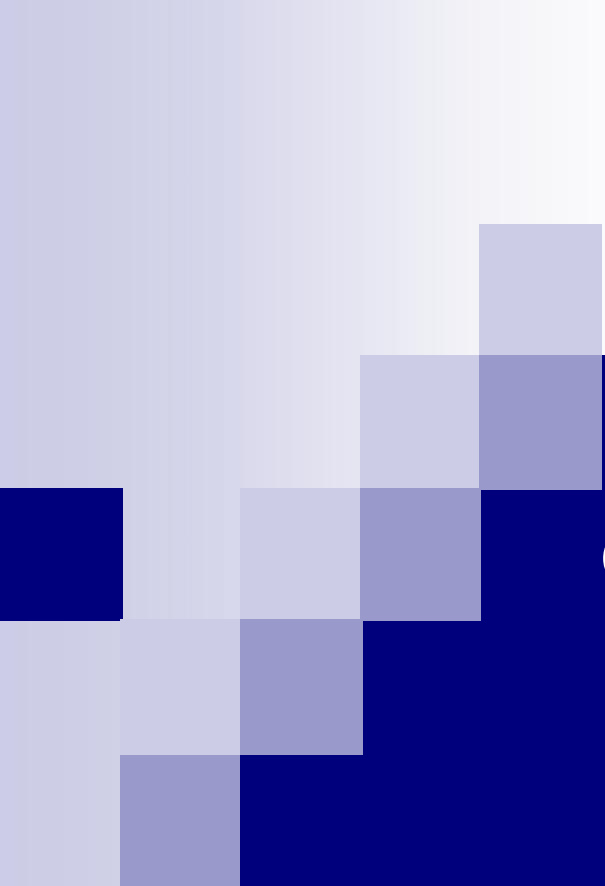




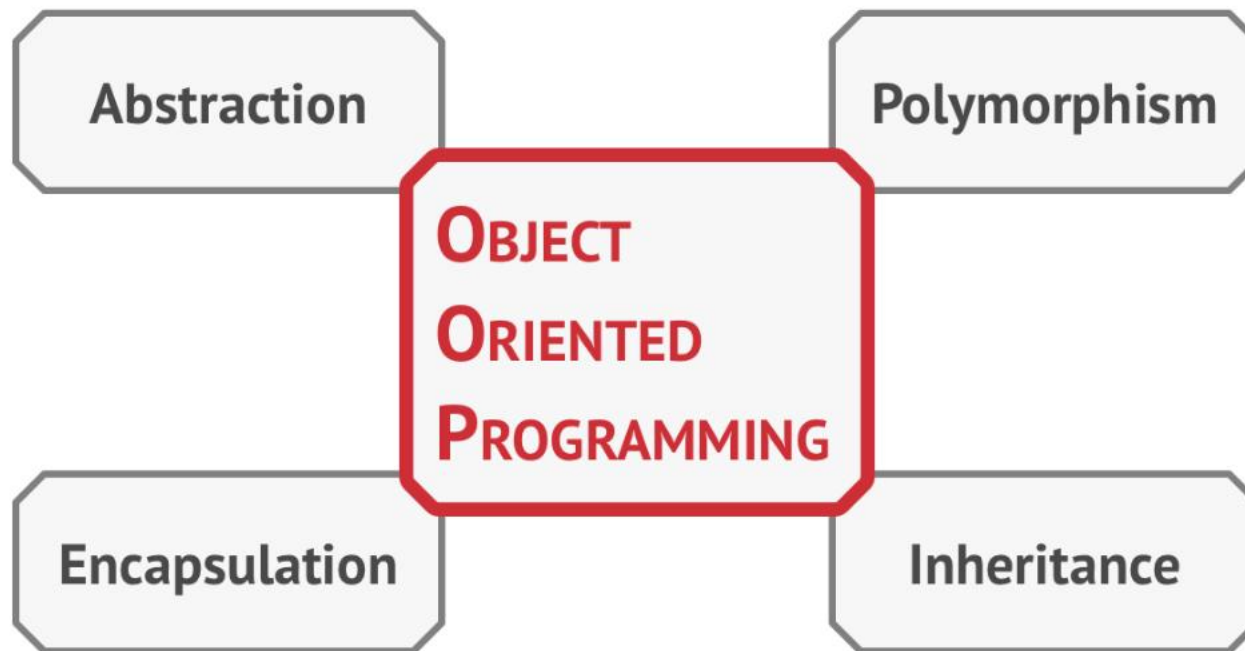
CPT304 – Object-oriented Concepts Software Engineering II Soon Phei, Tin



Topic

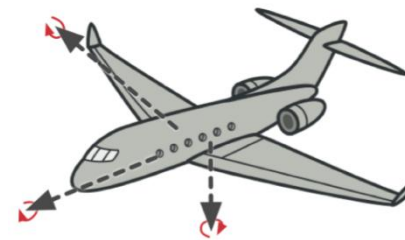
- OOP Recap
- What is a good design
- SOLID Principles
- Loosen coupling using composition

Four Pillar of OOP

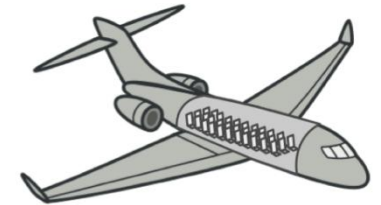


OOP - Abstraction

- Objects only *model* attributes and behaviors of real objects in a specific context, ignoring the rest.
- *Abstraction* is a model of a real-world object or phenomenon, limited to a specific context, which represents all details relevant to this context with high accuracy and omits all the rest.



Airplane
- speed - altitude - rollAngle - pitchAngle - yawAngle
+ fly()

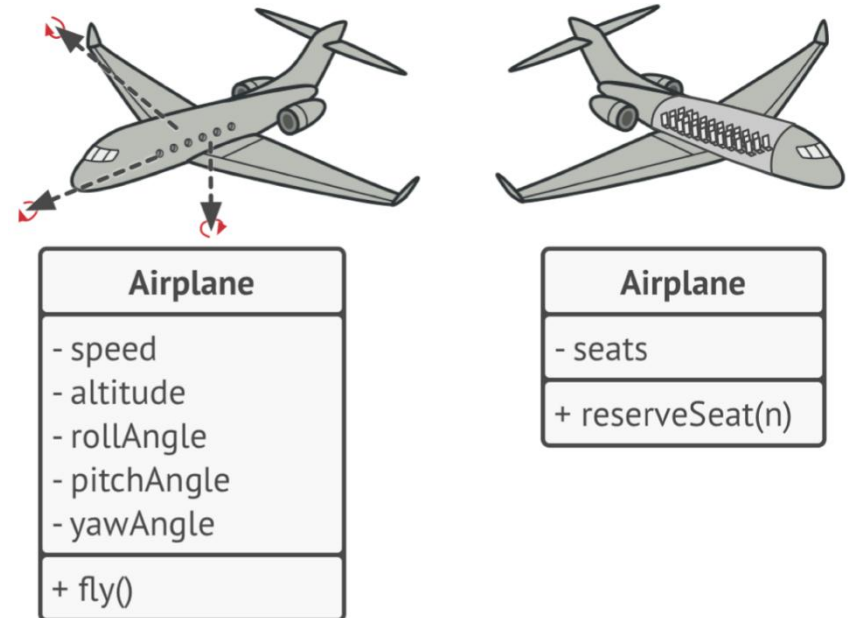


Airplane
- seats
+ reserveSeat(n)

Different models of the same real-world object.

OOP - Abstraction

- Abstraction allows developers to interact with a system at a higher level without needing to understand the intricate details of its implementation



Different models of the same real-world object.

OOP - Encapsulation

- The ability of an object to hide parts of its state and behaviors from other objects, exposing only a limited interface to the rest of the program.
- The interface mechanism lets you define contracts of interaction between objects.

Interface with different meaning: -

1. Public part of an object
2. Keyword in some programming language
3. The mean of communication between human and computer



OOP - Encapsulation

- The class that bind to the contract must implement the methods defined in the contract.
- When class X is working with other classes that binds to a contract, class X has definite confident that the classes it is working with have implemented methods defined in the contract.



OOP – Encapsulation

Exercise

- How do you create a contract in Java?
- How do you create a Java class that bind to a contract?



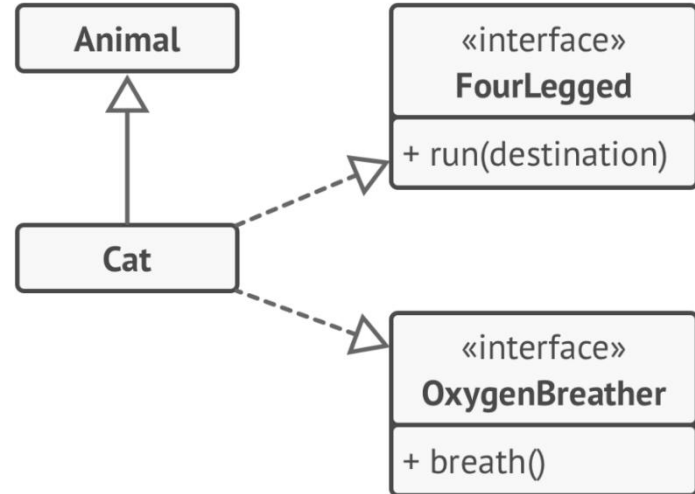
OOP - Inheritance

- The ability to build new classes on top of existing ones.
- The main benefit of inheritance is code reuse.
- Achieve code reuse by extending the existing class and put the extra functionality into a resulting subclass, which inherits fields and methods of the superclass.

OOP - Inheritance

- In most programming languages a subclass can extend only one superclass.
- On the other hand, any class can implement several interfaces at the same time.

- if a superclass implements an interface, subclasses inherit the implementation of the interface from its superclass.



UML diagram of extending a single class versus implementing multiple interfaces at the same time.



OOP - Polymorphism

- Allow a class to can takes different forms.
- The concept that you can access objects of different types through the same interface.
- Parent class reference is used to refer to a child class object.



Good Design



What is a good design?

- The one that hides inherent complexity and eliminates the accidental complexity
 - Hides inherent complexity
 - Abstraction, encapsulation, inheritance
 - Eliminate accidental complexity
 - Simplification, reusability, design patterns



What is a good design?

■ A software has a good design if the cost of changing it is minimum

■ How to achieve?

- Cohesion
 - The code is narrow and focus; It does only one thing and does it well.
- Coupling
 - The degree of dependency in the code



High Cohesion, Low Coupling

- Cohesion: High cohesion ensures that related functions stay together, which reduces the frequency of changes as the software evolves.
- Coupling: Low (loose) coupling prevents a change in one class from forcing cascading changes in others.



Highly cohesive code

- Code with similar/related (narrow and focus) functions stay together.
- For the software that evolve constantly, high cohesive code got to change less frequently
- For code with low cohesion
 - Every time you want to make a single small change (e.g., adding a new tax type), you have to go into multiple different classes and make small edits in all of them.



Loose coupling code

- Loose(low) coupling, instead of No coupling.
Impossible to have no dependency.
- Make the coupling loose.
- Depending on a class is tight(high) coupling.
- Depending on an interface is loose coupling.

Tight coupling code

```
5  public class Order {  
6      private Date orderDate;  
7      private double orderTotal;  
8      private Delivery del;  
9  
10     public double calculateTotal(){  
11         return orderTotal + del.getExpressCharge();  
12     }  
13 }
```

```
public class Delivery {  
    public double getExpressCharge(){  
        double charge = 0.00;  
  
        /*  
        charge = .....  
  
        */  
  
        return charge;  
    }  
}
```

Loose coupling code

```
5 public class Order {
6     private Date orderDate;
7     private double orderTotal;
8     private Delivery del = new ChinaDelivery();
9
10    public Order (Delivery del){
11        this.del = del;
12    }
13
14    public double calculateTotal(){
15        return orderTotal + del.getExpressCharge();
16    }
17 }
```

```
3 public interface Delivery {
4     public double getExpressCharge();
5 }
```

```
3 public class ChinaDelivery implements Delivery{
4
5     @Override
6     public double getExpressCharge() {
7         double charge = 0.00;
8         /*
9         charge = ..... Charges according to CHINA's rate and regulation
10        */
11        return charge;
12    }
13 }
```

```
3 public class BritishDelivery implements Delivery{
4
5     @Override
6     public double getExpressCharge() {
7         double charge = 0.00;
8         /*
9         charge = ..... Charges according to British's rate and regulation
10        */
11        return charge;
12    }
13 }
14 }
```

Exercise

- Modify the following code to improve its cohesion

```
public class Staff {  
    public Speaker saveSpeaker(Speaker speaker);  
  
    public boolean removeSpeaker(int speakerId);  
  
    public Speaker findById(int speakerId);  
  
    public void updateSpeakerPhoto(int speakerId, String photoUrl);  
  
    public Product saveProduct(Product product);  
  
    public void deleteProducts(List<Integer> productsId);  
  
    public boolean deleteProduct(int productId);  
}
```

Exercise

- Modify the following code to lower its coupling

```
public class ParttimeStaff {
    public double getWorkedHours(){
        ....
    }
}

public class Payroll {
    private ParttimeStaff staff;

    public Payroll(ParttimeStaff ps){
        this.staff = ps;
    }

    public double CalculatePay(){
        return staff.getWorkedHours() * .....|
    }
}
```

```
public class Car {
    public void move() {
        System.out.println("Car is moving");
    }
}

class Traveler {
    Car c = new Car();
    public void startJourney() {
        c.move();
    }
}
```

SOLID Principles

SOLID is a mnemonic for five design principles intended to make software designs more understandable, flexible and maintainable.

The cost of applying these principles into a program's architecture might be making it more complicated than it should be.

Striving for these principles is good, but always try to be pragmatic.

SOLID

Principles

S – Single Responsibility Principle

O – Open/closed Principle

L – Liskov Substitute Principle

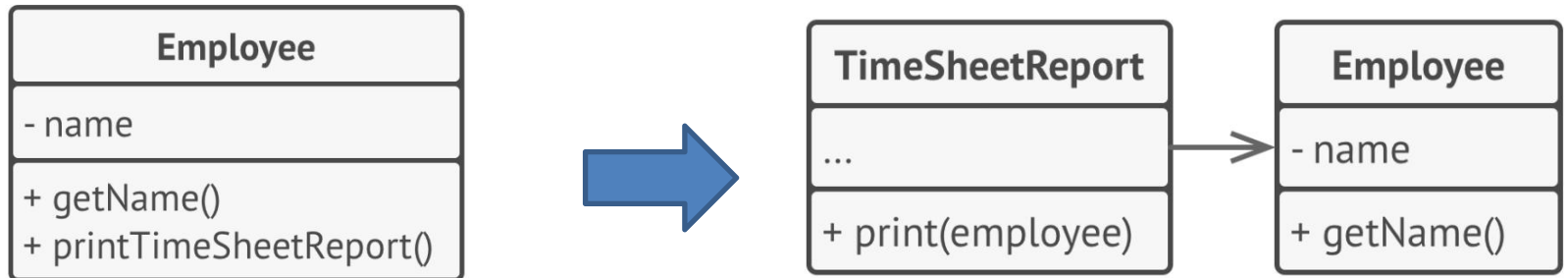
I – Interface Segregation Principle

D – Dependency inversion Principle

S – Single Responsibility Principle

- *A class should have one, and only one, reason to change.*
- *Make every class to have single responsibility, and make that responsibility entirely encapsulated by the class.*
- *The main goal of this principle is reducing complexity.*
- *A class with many responsibilities: -*
 - is more difficult to understand
 - often clutter with codes
 - difficult to navigate and hard to find a specific code
 - change to one feature might accidentally break another feature inside the same class

Example 6



BEFORE: the class contains several different behaviors.

AFTER: the extra behavior is in its own class.

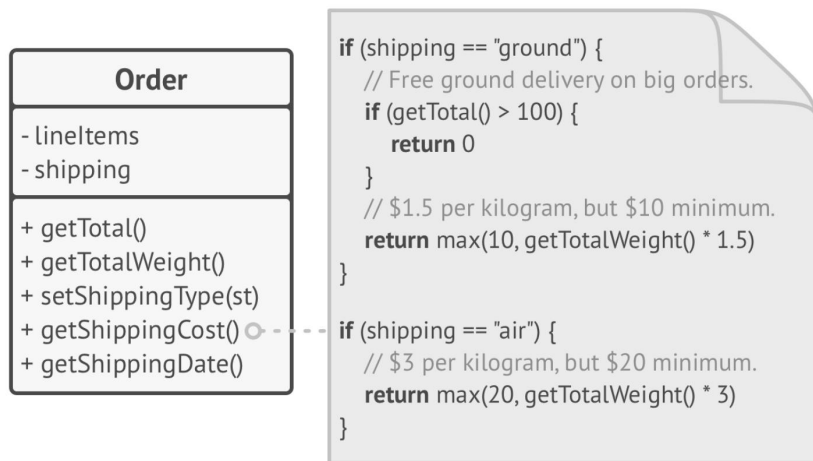
O – Open/Closed Principle

- *Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.*
- *The main idea of this principle is to keep existing code from breaking when you implement new features.*

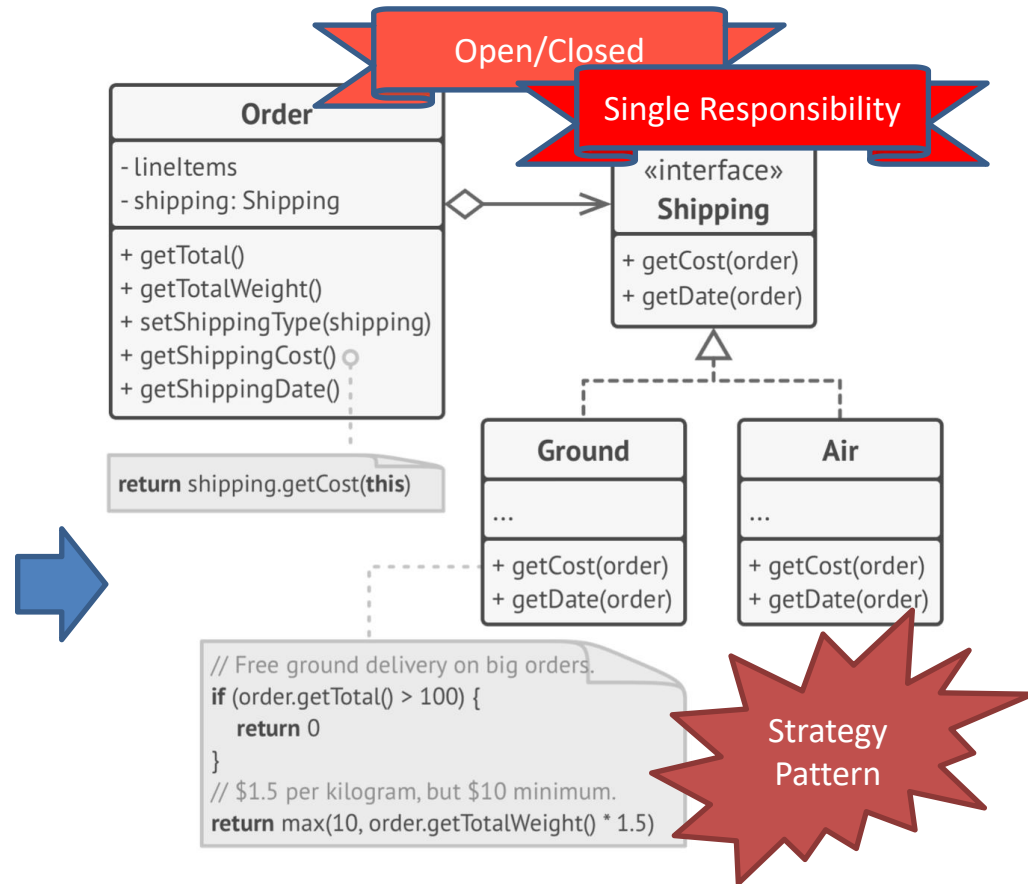
O – Open/Closed Principle

- *We can achieve Open/Closed Principle using inheritance. However, inheritance introduces tight coupling if the subclasses depend on implementation details of their parent class.*
- *Use interfaces instead of superclasses to allow different implementations without changing the code that uses them.*
- *The interfaces are closed for modifications, and you can provide new implementations to extend the functionality of your software.*

Example 7



*BEFORE: you have to change the **Order** class whenever you add a new shipping method to the app.*



AFTER: adding a new shipping method doesn't require changing existing classes.

Exercise

You are provided with the following class design for a simple notification system. The class below does not adhere to the SOLID principles, particularly the Single Responsibility Principle (SRP) and the Open/Closed Principle (OCP).

```
1 public class NotificationService {
2     public void sendEmail(String recipient, String subject, String body) {
3         // Code to send an email
4     }
5
6     public void sendSMS(String phoneNumber, String message) {
7         // Code to send an SMS
8     }
9
10    public void sendPushNotification(String deviceToken, String message) {
11        // Code to send a push notification
12    }
13
14    public void logNotification(String type, String message) {
15        // Code to log the notification details
16    }
17 }
```

Exercise

Tasks:

- *Redesign the class structure to ensure that each class has only one reason to change, adhering to the Single Responsibility Principle.*
- *Modify your design to comply with the Open/Closed Principle, allowing the system to support new notification methods without modifying existing code.*

L – Liskov Substitution Principle

Inheritance should be used only for substitutability

If an object of B should be used anywhere an object of A is used, then use inheritance.

If an object of B should use an object of A, then use composition.

Inheritance demands more from a developer than composition does

Services of the derived class should require no more and promise no less than the corresponding services of the base class.

Why?

The user of a base class should be able to use an instance of a derived class without knowing the difference.

L – Liskov Substitution Principle

- *When extending a class, remember that you should be able to pass objects of the subclass in place of objects of the parent class without breaking the client code.*
- *This means that the subclass should remain compatible with the behavior of the superclass.*

L – Liskov Substitution Principle

Rule 1 - Parameter types in a method of a subclass should match or be more abstract (more general) than parameter types in the method of the super- class.

Rule 2 - The return type in a method of a subclass should match or be a subtype (more specific) of the return type in the method of the superclass.

```
public abstract class Foo {  
    public abstract Number generateNumber();  
    // Other Methods  
}
```



Valid

```
public class Bar extends Foo {  
    @Override  
    public Integer generateNumber() {  
        return new Integer(10);  
    }  
    // Other Methods  
}
```

L – Liskov Substitution Principle

Rule 3 - A method in a subclass shouldn't throw types of exceptions which the base method isn't expected to throw.

types of exceptions should *match* or be *subtypes* of the ones that the base method is already able to throw.

Rule 4 - A subclass shouldn't strengthen pre-conditions.

A subclass shouldn't override some methods and strengthen/increase the pre-condition.

```
public class Foo {  
    // precondition: 0 < num <= 5  
    public void doStuff(int num) {  
        if (num <= 0 || num > 5) {  
            throw new IllegalArgumentException("Input out of range 1-5");  
        }  
        // some logic here...  
    }  
}
```

Valid

```
public class Bar extends Foo {  
    @Override  
    // precondition: 0 < num <= 10  
    public void doStuff(int num) {  
        if (num <= 0 || num > 10) {  
            throw new IllegalArgumentException("Input out of range 1-10");  
        }  
        // some logic here...  
    }  
}
```

L – Liskov Substitution Principle

Rule 5 - A subclass shouldn't weaken post-conditions.

A subclass shouldn't override some methods and weaken/reduce the post-condition.

```
public abstract class Car {  
  
    protected int speed;  
  
    // postcondition: speed must reduce  
    protected abstract void brake();  
  
    // Other methods...  
}
```



Valid

```
public class HybridCar extends Car {  
  
    // Some properties and other methods...  
  
    @Override  
    // postcondition: speed must reduce  
    // postcondition: charge must increase  
    protected void brake() {  
        // Apply HybridCar brake  
    }  
}
```

L – Liskov Substitution Principle

Rule 6 - Invariants of a superclass must be preserved.

- This is probably the least formal rule of all.
- A class invariant is an assertion concerning object properties that must be true for all valid states of the object.
- The rule on invariants is the easiest to violate because you might misunderstand or not realize all of the invariants of a complex class.
- The safest way to extend a class is to introduce new fields and methods, and not to modify with any existing members of the superclass.

```
public abstract class Car {  
    protected int limit;  
  
    // invariant: speed < limit;  
    protected int speed;  
  
    // postcondition: speed < limit  
    protected abstract void accelerate();  
  
    // Other methods...  
}
```



Valid

```
public class HybridCar extends Car {  
    // invariant: charge >= 0;  
    private int charge;  
  
    @Override  
    // postcondition: speed < limit  
    protected void accelerate() {  
        // Accelerate HybridCar ensuring speed < limit  
    }  
  
    // Other methods...  
}
```

L – Liskov Substitution Principle

Rule 7 - A subclass shouldn't change values of private/protected states of the superclass

```
public abstract class Car {  
  
    // Allowed to be set once at the time of creation.  
    // Value can only increment thereafter.  
    // Value cannot be reset.  
    protected int mileage;  
  
    public Car(int mileage) {  
        this.mileage = mileage;  
    }  
  
    // Other properties and methods...  
}
```



Invalid

```
public class ToyCar extends Car {  
    public void reset() {  
        mileage = 0;  
    }  
  
    // Other properties and methods  
}
```

L – Liskov Substitution Principle

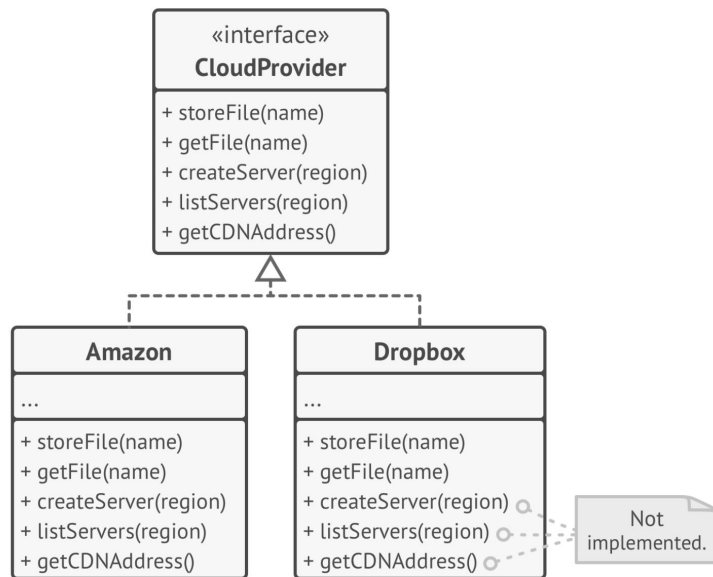
Why LSP is important?

- Code that adhere to LSP is flexible and promote high degree of reusability.
- Code that violate LSP is tightly coupled and creates unnecessary entanglements.
- If client code cannot substitute a superclass reference with a subclass object freely, it would be forced to do ***instanceof*** checks and specially handle some subclasses.
- If this kind of conditional code is spread across the codebase, it will be difficult to maintain.
- Code that adheres to the LSP is code that makes the right abstractions.

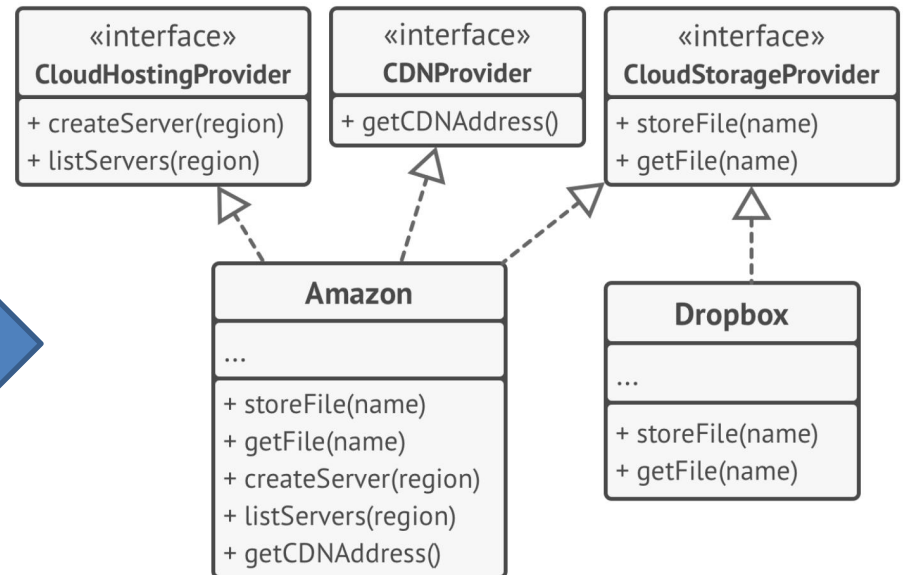
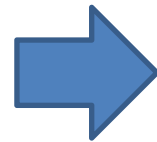
I – Interface Segregation Principle

- *Clients should not be forced to depend upon interfaces that they do not use.*
- *Make your interface narrow enough that client don't need to implements the behaviors they don't need.*
- *This principle is easy to violate, especially if your software evolves and you have to add more and more features over time.*
- *Like the Single Responsibility Principle, the goal of this principle is to reduce complexity and hence the side effects and frequency of required changes by splitting the software into multiple, independent parts.*

Example 8



BEFORE: not all clients can satisfy the requirements of the bloated interface.



AFTER: one bloated interface is broken down into a set of more granular interfaces.

D – Dependency Inversion Principle

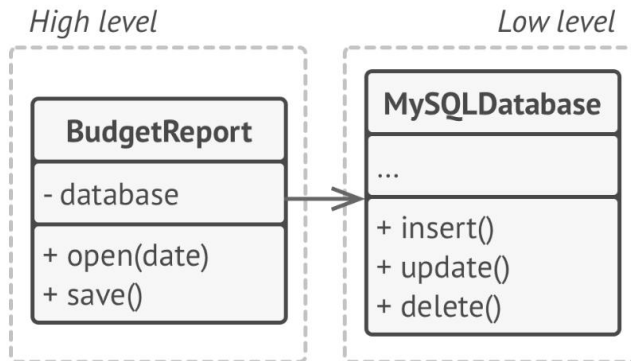
- *High-level modules, which provide complex logic, should be easily reusable and unaffected by changes in low-level modules, which provide utility features.*
- *To achieve that, you need to introduce an abstraction that decouples the high-level and low-level modules from each other.*
- *Definition of the Dependency Inversion Principle consists of two parts:*
 - High-level modules should not depend on low-level modules. Both should depend on abstractions.
 - Abstractions should not depend on details. Details should depend on abstractions.

D – Dependency Inversion Principle

- ***Low-level classes** implement basic operations such as working with a disk, transferring data over a network, connecting to a database, etc.*
- ***High-level classes** contain complex business logic that directs low-level classes to do something.*
- *The dependency inversion principle suggests to work on the high-level classes first.*
- *Complete the high-level classes by dependent on abstraction of the low-level classes (the interface).*
- *Then complete the low-level classes by writing the concrete classes of the abstraction (implementing the interface).*

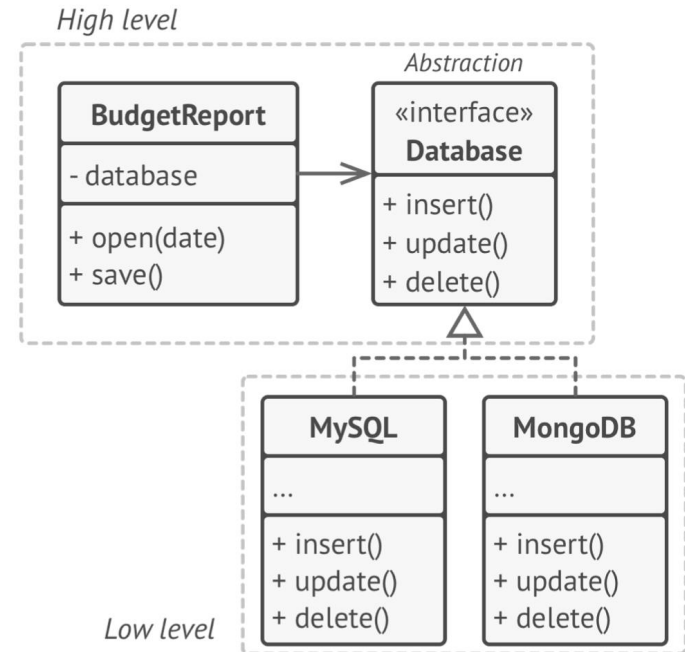
Example 9

Complete low-level class first



BEFORE: a high-level class depends on a low-level class.

Complete high-level class first



AFTER: low-level classes depend on a high-level abstraction.

Exercise

1. *According to SOLID principles, which is the most suitable situation to use inheritance?*
2. *According to SOLID principles, which is the most suitable situation to use composition?*
3. *Explain in detail the problem when the return type in an override method of a subclass is a long data type while the return type in the method of the superclass is an int data type?*
4. *Discuss the Single Responsibility Principle (SRP) and provide an example of how it can be applied in a software design.*

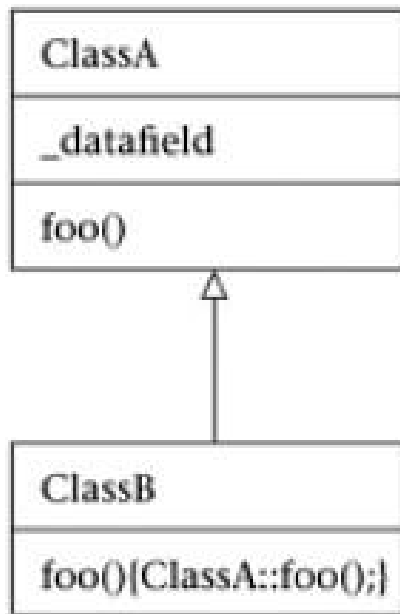
Exercise

5. *How does the Open/Closed Principle (OCP) contribute to software maintainability and extensibility? Provide a real-world example to illustrate this principle.*
6. *Explain the Interface Segregation Principle (ISP) and discuss its role in software design.*
7. *Describe how the Dependency Inversion Principle (DIP) helps to decouple software modules and enhance flexibility in a software architecture.*

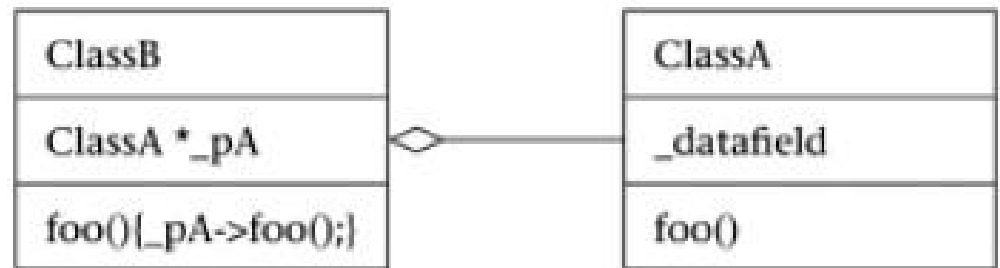


Loosen Coupling using Composition

Composition (has a) and Inheritance (is a)



Class B is a Class A



Class B has a Class A

You can always replace an inheritance relationship by a composition relationship

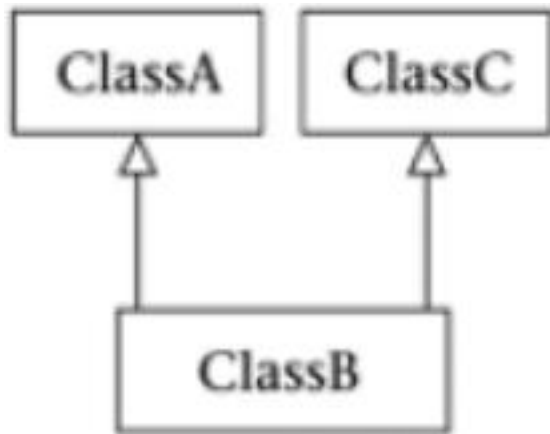


Usage of Composition

- Use composition as an alternative to multiple inheritance
- **Dynamic Flexibility:** Composition makes dynamic changes possible at runtime (often through polymorphism), allowing an object to change its behavior by swapping out its composed parts.
- **Avoids Combinatorial Explosion:** Composition allows you to combine behaviors dynamically without creating a new class for every possible combination of traits.

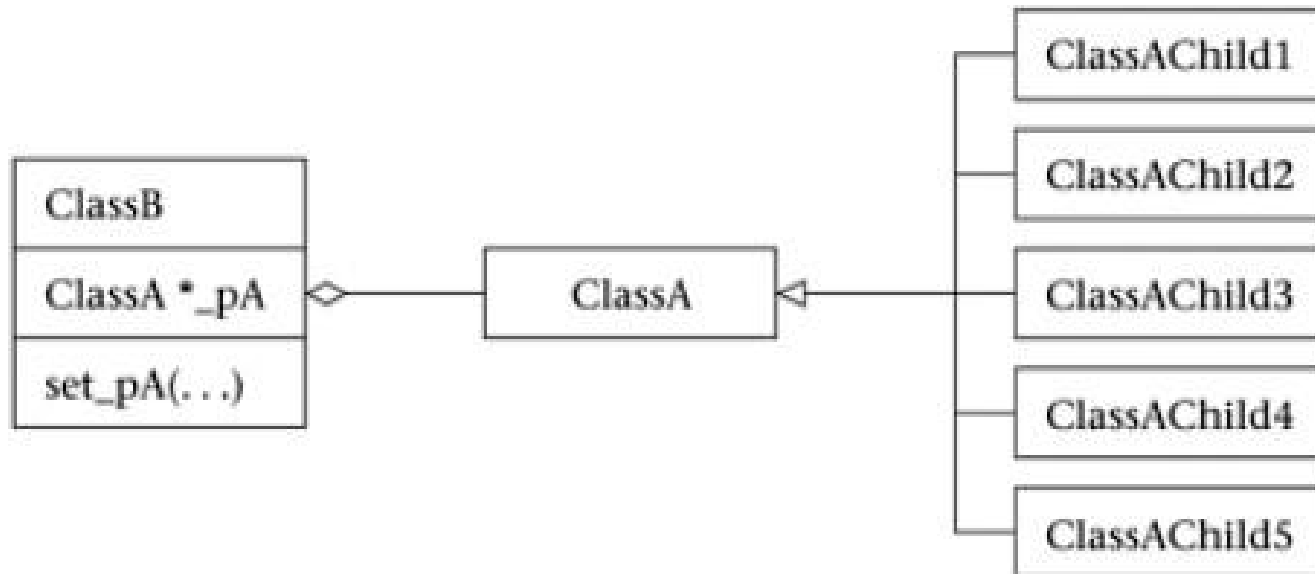
Usage of Composition

- Use composition as an alternative to multiple inheritance

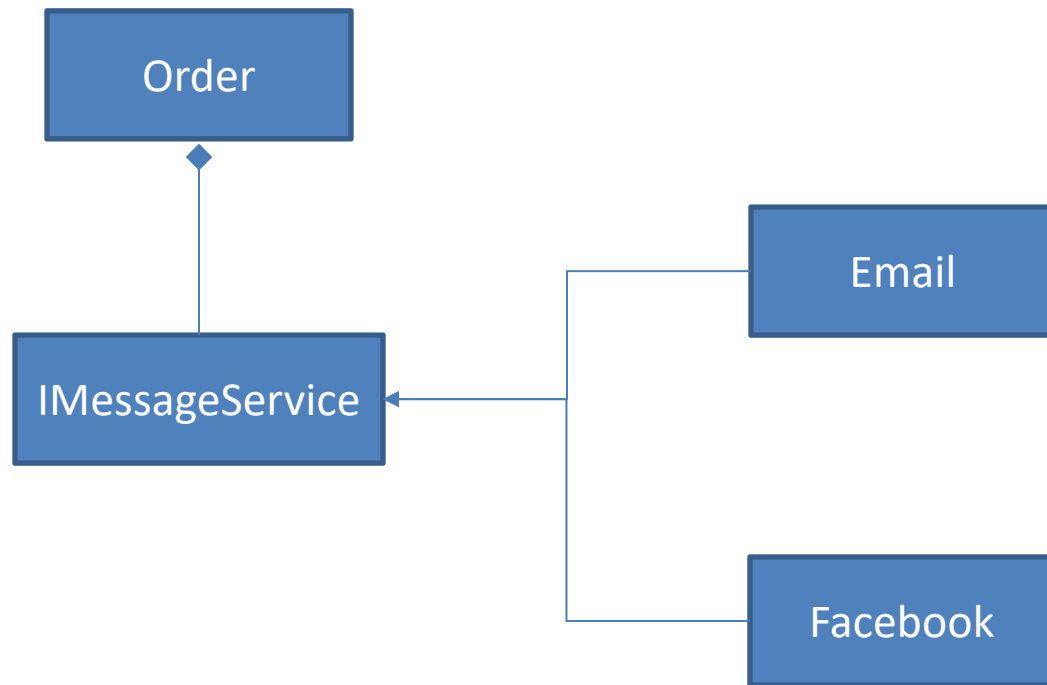


Usage of Composition

- Composition *makes dynamic change possible*
- Think about polymorphism



Example



```
1 interface IMessageService{
2     public void Send(String text);
3 }
```

```
1 class Email implements IMessageService{
2     public void Send(String text){
3         System.out.println(text + "\nSending produ
4     }
5 }
```

```
1 class Facebook implements IMessageService{
2     public void Send(String text){
3         System.out.println(text + "\nSending produ
4     }
5 }
```

```
1 class Order{
2     private IMessageService ms;
3     private Customer c;
4
5     public Order(Customer c) {
6         this.c = c;
7         if(c.IsPreferEmail()){
8             ms = new Email();
9         }else{
10             ms = new Facebook();
11         }
12     }
13
14     public void SendMessage(){
15         ms.Send("Dear " + c.GetName() + ", thank you ...");
16     }
17 }
```

```
1 class OrderServiceComposition{
    Run | Debug
2     public static void main(String args[]){
3         Order o = new Order(new Customer("David", false));
4
5         o.SendMessage();
6     }
7 }
```

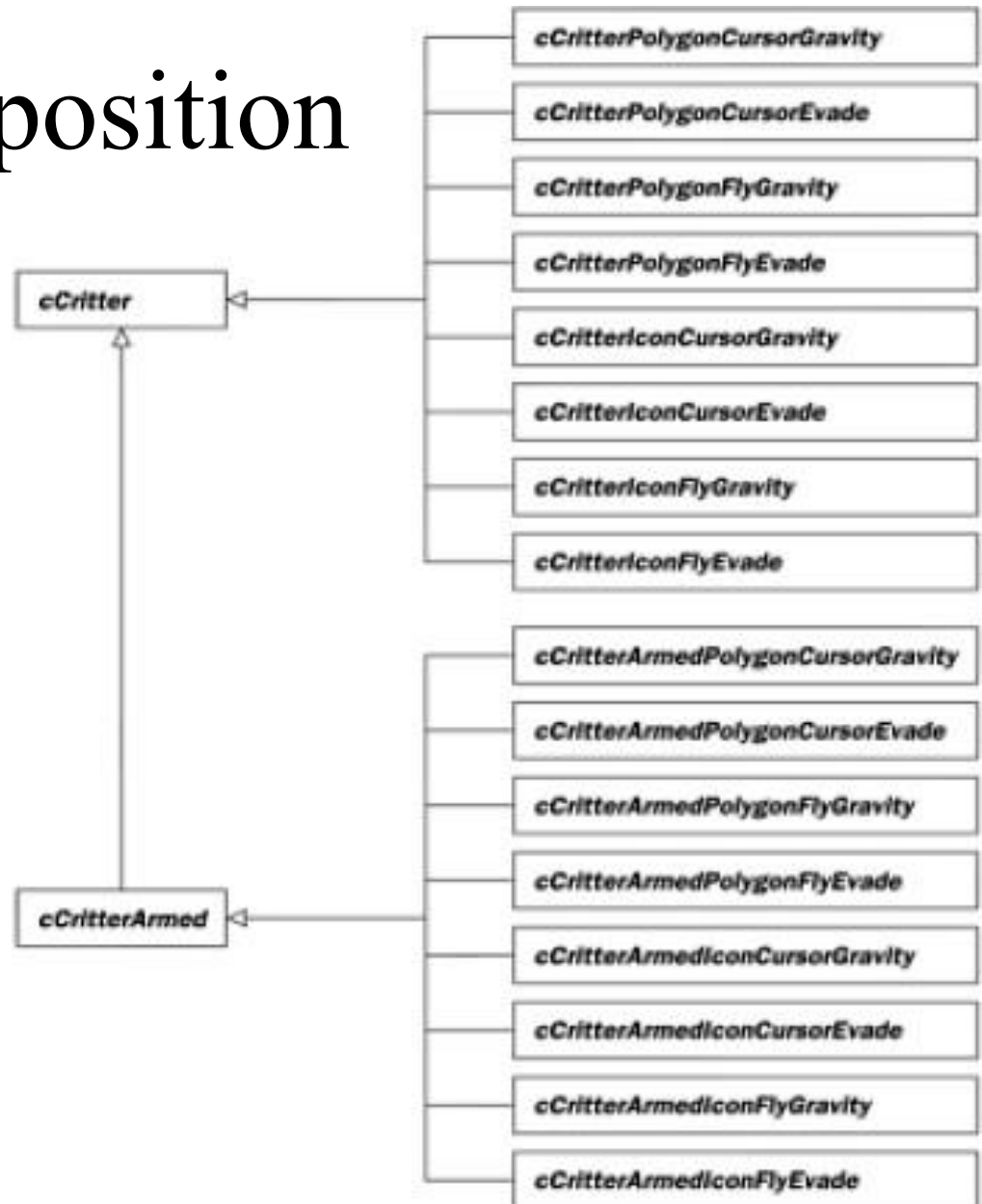


Usage of Composition

- Combinatorial Explosion
- Occurs when inheritance is used to model multiple independent traits
- Increase in the number of subclasses required to cover every possible combination of those traits

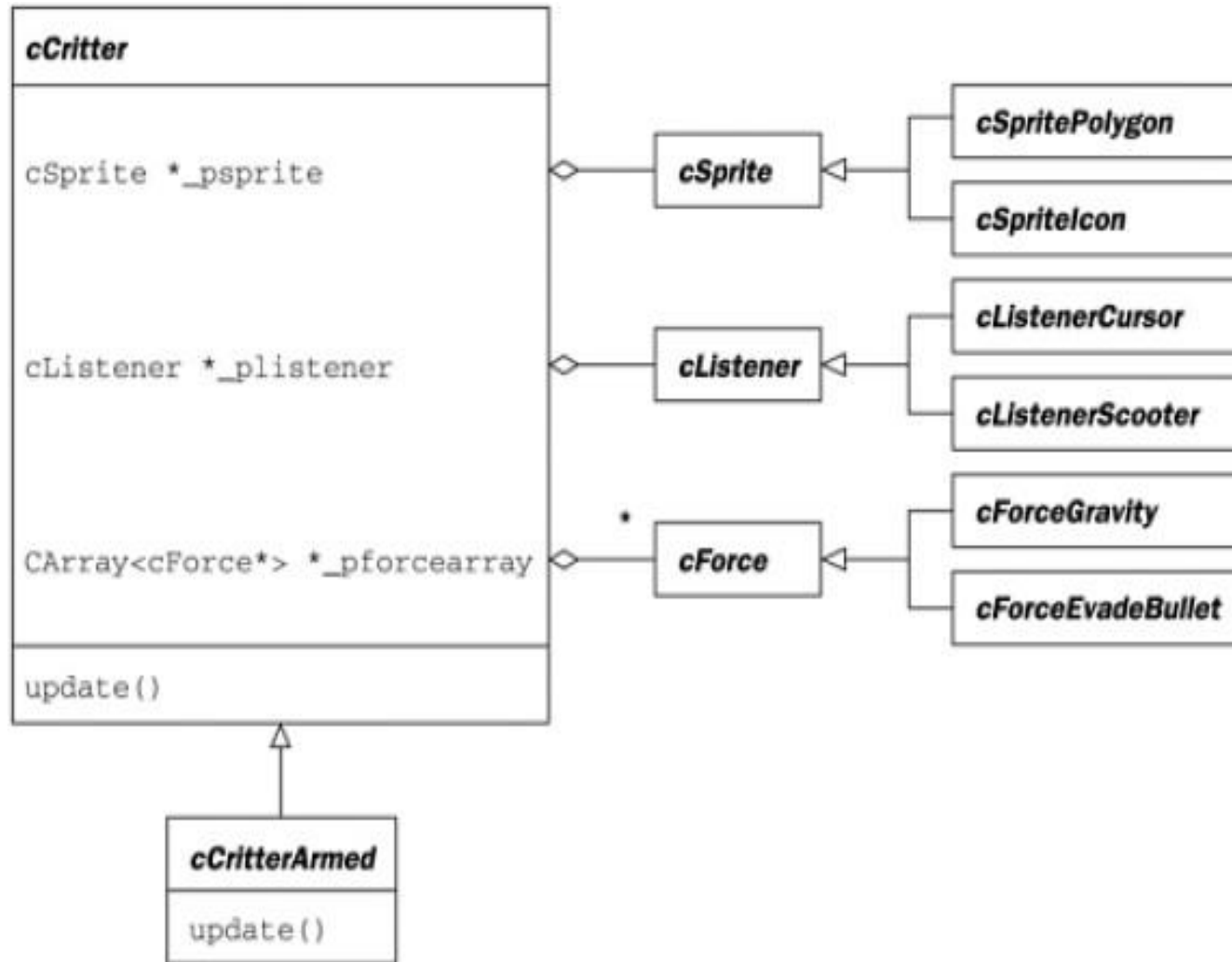
Usage of Composition

- Use composition to avoid the 'combinatorial explosion'



Usage of Composition

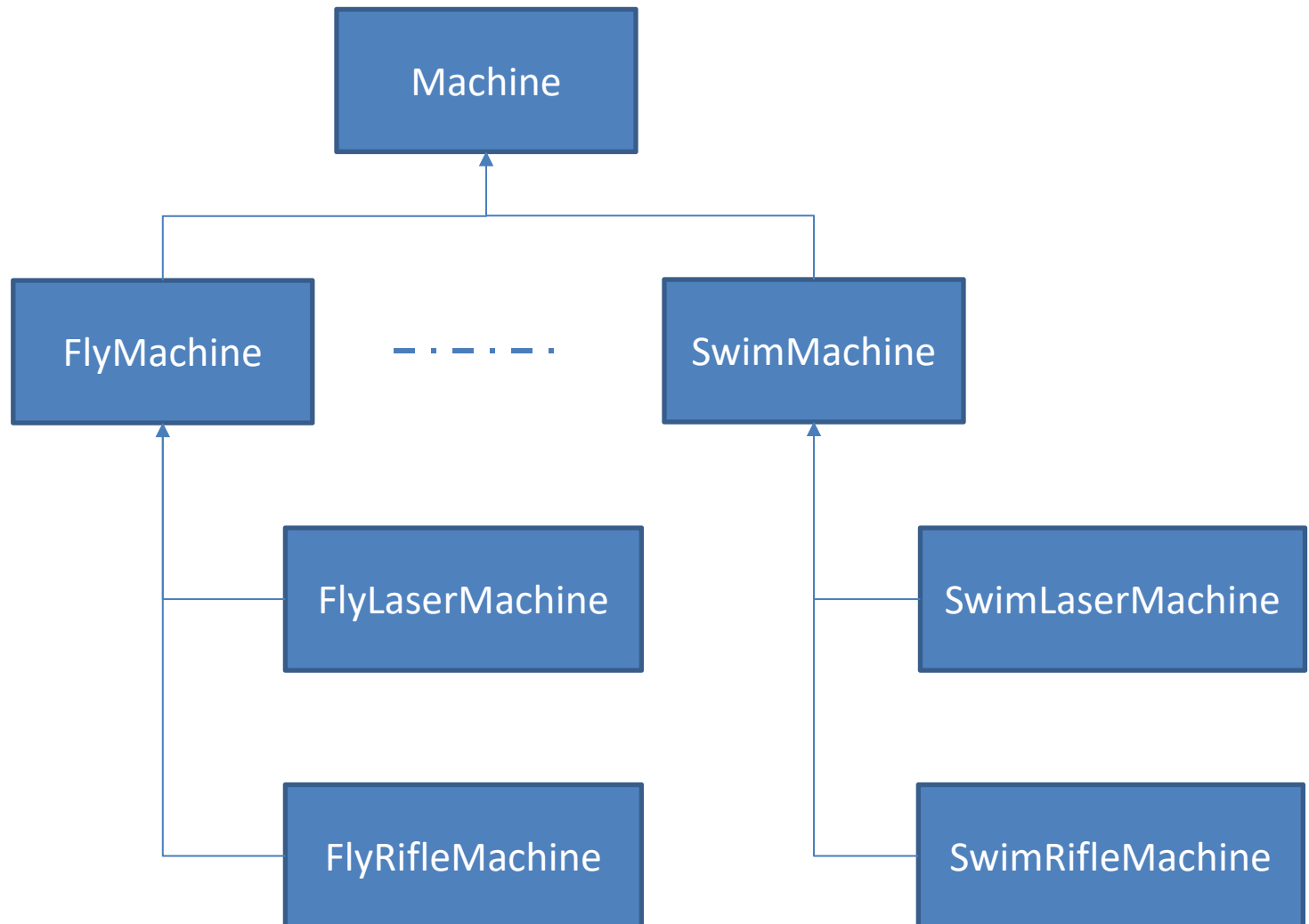
- Use composition to avoid the 'combinatorial explosion'



Machines

There are several types of machines. Some can fly and some can swim. Regardless of how the machine move, they can attack enemy by using either laser or rifle. Some machines does not have the attack capability.

Inheritance Hierarchy



```

1  abstract class Machine{
2      private int currentX;
3      private int currentY;
4      private int speed;
5
6      public Machine(int currentX, int currentY, int speed){
7          this.currentX = currentX;
8          this.currentY = currentY;
9          this.speed = speed;
10     }
11
12     public abstract void Go(int X, int Y);
13 }

```

```

1  class SwimMachine extends Machine{
2      private int depth;
3
4      public SwimMachine(int depth, int currentX, int currentY, int speed){
5          super(currentX, currentY, speed);
6          this.depth = depth;
7      }
8
9      public void Go(int X, int Y){
10         System.out.println("Swim to set destination!");
11     }
12 }

```

```

1  class FlyMachine extends Machine{
2      private int altitude;
3
4      public FlyMachine(int altitude, int currentX, int currentY, int speed){
5          super(currentX, currentY, speed);
6          this.altitude = altitude;
7      }
8
9      public void Go(int X, int Y){
10         System.out.println("Fly to set destination!");
11     }
12 }

```

```
1 class FlyLaserMachine extends FlyMachine{
2     private boolean charging;
3     private int chargeDuration;
4     private int aimX;
5     private int aimY;
6     private int angle;
7     private int damage;
8
9     public FlyLaserMachine(int altitude, int currentX, int currentY, int speed, int damage, int chargeDuration){
10         super(altitude, currentX, currentY, speed);
11         this.charging = false;
12         this.chargeDuration = chargeDuration;
13         this.damage = damage;
14     }
15
16     public void Aim(int aimX, int aimY, int angle){
17         this.aimX = aimX;
18         this.aimY = aimY;
19         this.angle = angle;
20     }
21
22     public void Kill(){
23         if(!this.charging)
24             System.out.println("Fire my LASER!!!");
25     }
26 }
```

Machine Classes with Composition

How can we achieve the same features using Composition instead of Inheritance?

```

1  public class Machine {
2      private Weapon weapon;
3      private Movement movement;
4
5      public Machine(Weapon w, Movement m){
6          this.weapon = w;
7          this.movement = m;
8      }
9
10     public void attack(){
11         weapon.shot();
12     }
13
14     public void moveToPosition(int x, int y){
15         movement.setPositionX(x);
16         movement.setPositionY(y);
17         movement.move();
18     }
19 }

```

```

1  public interface Weapon {
2      public double shot();
3  }

```

```

1  public abstract class Movement {
2      private int x, y;
3
4      public void setPositionX(int x){
5          this.x = x;
6      }
7
8      public void setPositionY(int y){
9          this.y = y;
10     }
11
12     public int getPositionX(){
13         return x;
14     }
15
16     public int getPositionY(){
17         return y;
18     }
19
20     public abstract void move();
21 }

```

```
1 public class Rifle implements Weapon {
2     @Override
3     public double shot(){
4         // You could implement code that specific to shoting
5
6         System.out.println(x:"I am shoting using Rifle");
7         return 1.5;
8     }
9 }
```

```
1 public class Jump extends Movement{
2
3     @Override
4     public void move() {
5         // You could implement code that specific to jumping
6
7         System.out.println("I am JUMPING to (" + super.getPositionX() +
8             "," + super.getPositionY() +
9             ")");
10    }
11
12 }
```

End